Speculative Decoding and MultiToken-Pipelines

Tommy Long, Angel Mendez, Nathan Lai

Motivation and Problem: One Token at a Time Bottleneck

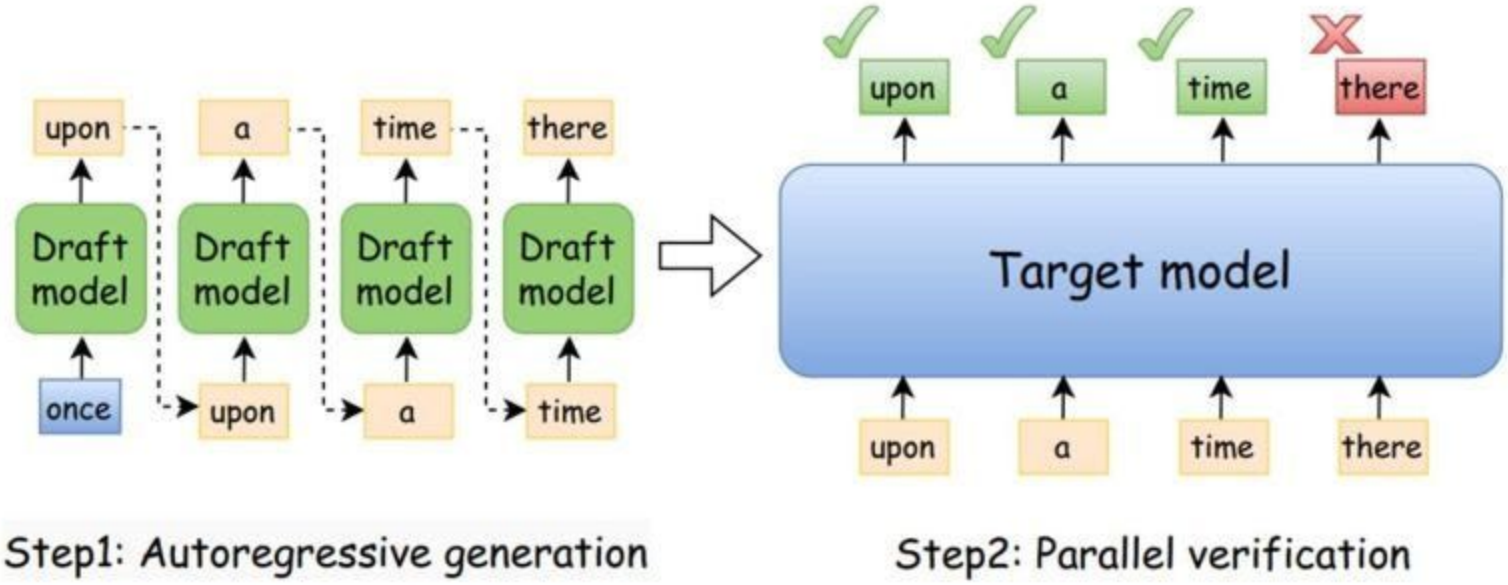
1. Autoregressive decoding can’t generate the next token until the current token is generated
   - Each token depends on the previous tokens
2. For every forward pass, the model needs to be loaded into memory which is a bottleneck with larger models

- The bottleneck is the memory bounds and previous token dependency
- How can we better utilize hardware to workaround the problem?
  - GPU is underutilized, memory is the bottleneck

What is Speculative Decoding?

- Speculative decoding generates multiple tokens at once
- Employs two models: a draft and target model
  - Draft model proposes K draft tokens based on input sentence
  - Target model takes input sentence and draft tokens and verifies the draft tokens in one single forward pass
- Target model accepts or rejects draft tokens
  - If token is accepted, it is added to input sentence and the next draft token is checked
  - If token is rejected, all subsequent draft tokens are rejected
  - Either way, a bonus token is generated either at the end or first disagreement
- By pushing all the draft tokens into one pass for the target model, we use more computational strength with less memory usage

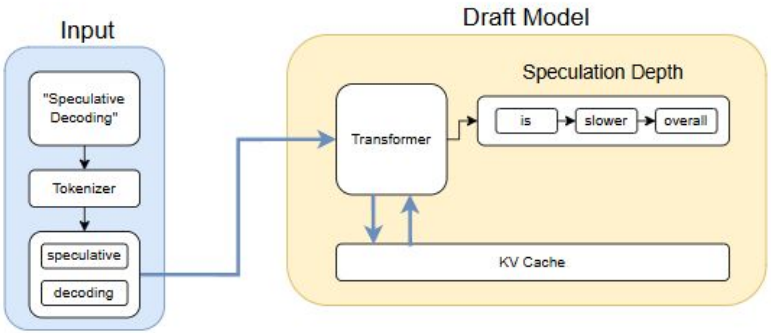
Token Timeline (Speculation vs Verification)



https://medium.com/@genai.works/speed-up-llm-inference-with-speculative-decoding-1fc79701e9d6

Speculation Depth (k)

- k = Speculation Depth = How many draft tokens to generate before verification
- Large speculation depths
  - More tokens accepted in one forward pass if target model is confident
  - If a single draft token is rejected, all preceding draft tokens are discarded
  - Risk of low acceptance rate
- Small speculation depths
  - If draft token is rejected, lose only a few draft tokens
  - Chance of higher acceptance rate



Acceptance Rate

$$\text{Acceptance Rate} = \alpha = \frac{\text{Accepted Tokens}}{\text{Total draft tokens}}$$

- High acceptance rates
  - Increase in speed as more tokens are accepted and generated
  - Lower number of rejections and rollbacks → Lower latency
- Low acceptance rates
  - More resampling
  - Higher number of rejections and rollbacks → Higher latency

Adaptive Speculation Depth

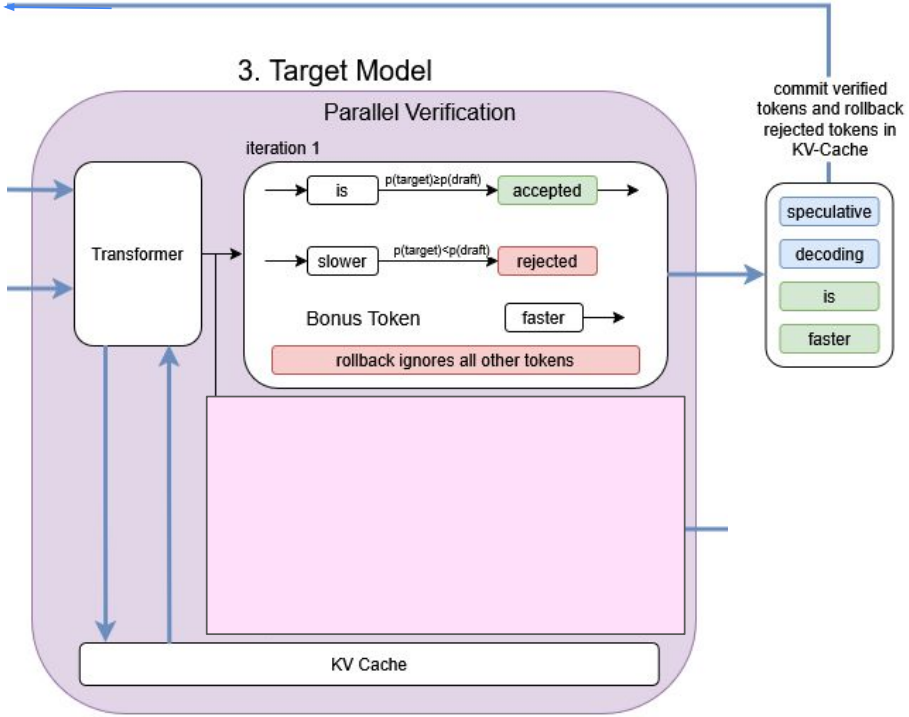
- Start with a baseline **k** as a ratio of sequence length **n** (10% of **n**)
- Adaptive Speculation Depth automatically finds the sweet spot without trial and error
  - If Acceptance Rate is high, we slowly increase k
  - If Acceptance Rate is low, we aggressively decrease k
- $k \geq 2$, otherwise there is no speculation
- Basically, if we are confident in the faster draft tokens, we verify more at once

Acceptance and Rejection Logic

- Commonly two ways: greedy (deterministic) or sampling (stochastic)
- Greedy
  - If draft token = verification token, ACCEPT
  - Else, reject
  - So, typically higher reject rate but very exact
- Sampling (chosen for our implementation)
  - Create an acceptance probability and randomly sample whether it accepts
  - Can get varied results from run to run, but higher acceptance rate
  - Still maintains target model output distribution
  - We used this to get a higher acceptance rate without loss in accuracy

Rollback Mechanism

- When a draft token is rejected, the corrected token is sampled from a probability distribution and added to the input sentence
  - All other draft tokens are discarded
- Loops back to draft token generation where draft model predicts the next K draft tokens based on the new input sentence



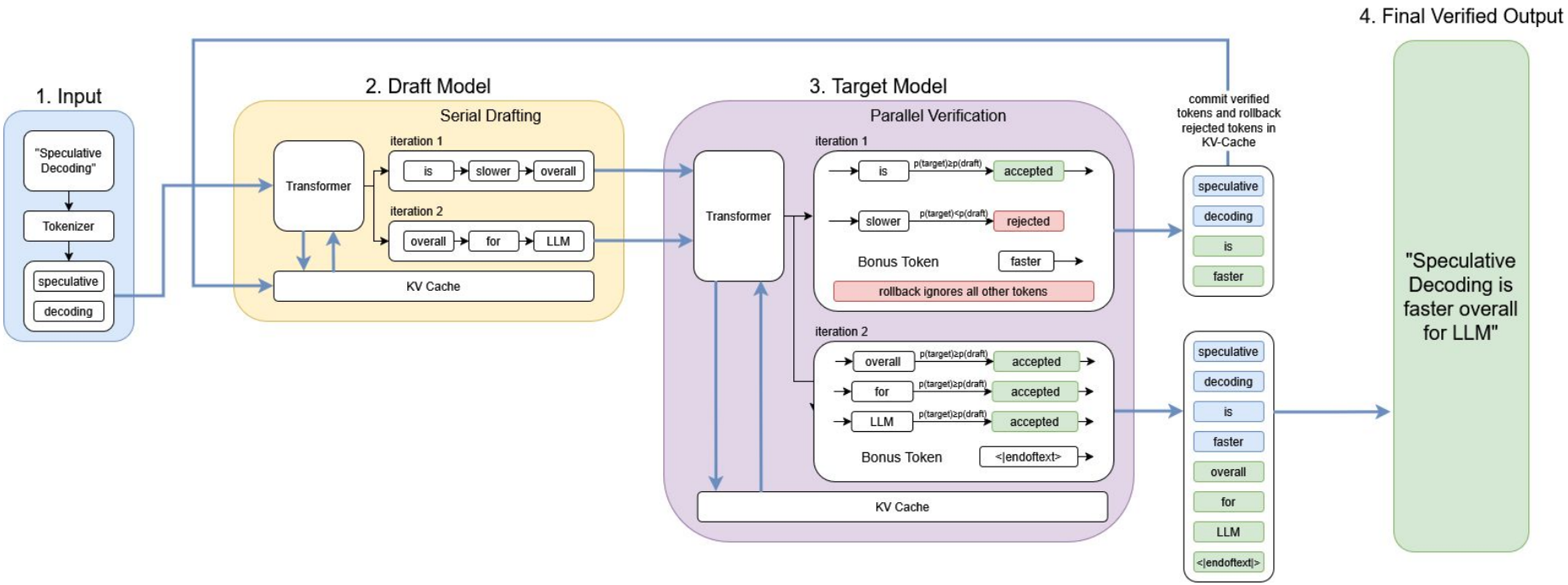
Hardware-Assisted Rollback

- Uses a speculative buffer to store draft tokens
- Maintains:
  - Commit pointer -> last verified token
  - Rollback pointer -> last verified token
- During verification:
  - Matching tokens are committed
  - On mismatch -> rollback triggered
- Rollback mechanism:
  - Discards all speculative tokens after mismatch
  - Avoids recomputation from scratch
- Improves:
  - Recover time after errors
  - Overall pipeline efficiency

Pipeline Stages (Draft -> Verify -> Commit)

- Draft Stage:
  - Draft model generates a speculative batch of k tokens
  - Batch is packaged with base context and metadata
- Verify Stage
  - Target model verifies the oldest batch in the queue
  - Tokens are accepted until the first mismatch
- Commit / Rollback Stage
  - Accepted prefix is committed to the output
  - On mismatch, the correct token is emitted
  - Younger speculative batches are flushed
- Repeat
  - Pipeline continues until n output tokens are generated or EOS is reached

System Model Overview



Multi-Token Pipeline Architecture

- Two-stage Pipeline:
  - Draft stage -> generates k speculative tokens
  - Verification stage -> validates tokens using target model
- Uses a Buffered Pipeline to Allow Overlap Between Stages
- While Verification is Running:
  - Draft model continues generating future tokens
- Maintains a Queue of Speculative Batches
- Ensures Correctness Via:
  - Commit (accepted tokens)
  - Rollback (on mismatch)

Pipeline Hazards and Challenges

- Rollback Hazard
  - Occurs when draft tokens are mismatch target output
  - Forces rollback and discards speculative work
- Pipeline Bubbles (Idle Cycles)
  - Occur when one stage waits for the other
  - Reduces overall pipeline utilization
- Speculative Waste
  - Incorrect draft tokens lead to wasted computation
  - More severe with low acceptance rate
- Buffer Imbalance
  - Overfilled buffer -> wasted speculative batches
  - Underfilled buffer -> underutilized pipeline

Buffering and Backpressure

- Uses a bounded buffer (queue) to store speculative batches
- Buffer Capacity Controls Parallelism
  - Large buffer -> more overlap between stages
  - Smaller buffer -> less speculative work
- Backpressure mechanism
  - Draft stage pauses when buffer is full
  - Prevents excessive speculative generation
- Trade-offs
  - Large buffer:
    - Higher parallelism
    - More wasted work on rollback
  - Small buffer:
    - Lower waste
    - More pipeline idle time

KV-Cache Design Overview

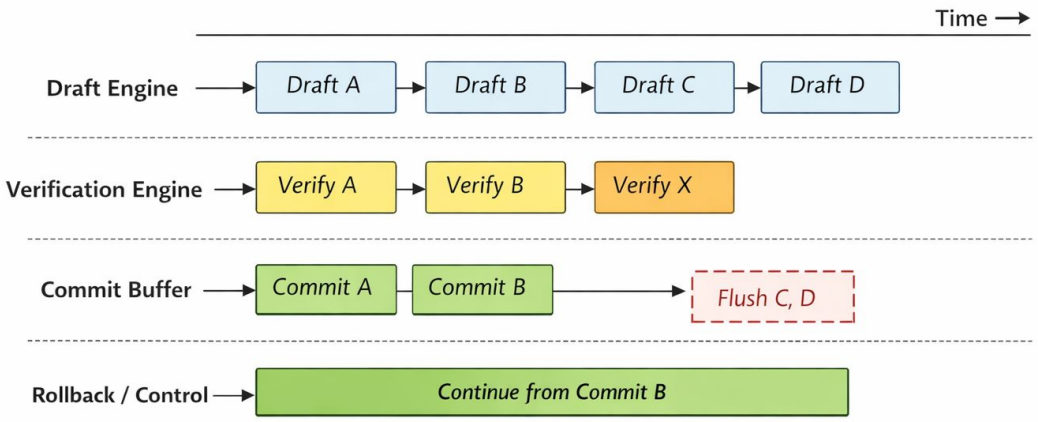
- Maintains two states:
  - committed_tokens -> verified output
  - speculative_tokens -> temporary draft predictions
- Draft tokens are stored in speculative state
- Verified tokens are committed to final output
- On mismatch:
  - speculative tokens are discarded
- Ensures final output matches baseline decoding

Speculative vs Committed KV States

- Committed State
  - Contains verified tokens
  - Represents final output sequence
  - Never modified after commit
- Speculative State
  - Contains draft tokens
  - Temporary and subject to verification
  - Can be discarded on mismatch
- State Transition
  - Draft tokens -> speculative state
  - Accepted tokens -> moved to committed state
  - Rejected tokens -> removed

KV-Cache Rollback and Consistency

- During verification:
  - Tokens are compared between draft and target models
- On match:
  - Tokens are committed to final output
- On mismatch
  - All speculative tokens are discarded
  - Correct token is inserted
- Consistency Guarantee
  - Only verified tokens are committed
  - Final output baseline autoregressive decoding



Implementation Overview

- Baseline Encoders
  - TinyLlama (Llama2-1.1b)
  - Llama2-7b
- Speculative Encoder
  - Draft: Llama2-1.1b
    - Smaller, faster model
  - Target: Llama2-7b
    - Larger, more accurate and generalized model
  - Adaptive Speculation Depth
  - KV-Cache
- Factual prompt for accuracy check
  - “The first digits of pi are”

Theoretical Performance Model

1. cost ratio = $\beta = \dfrac{\text{draft model time}}{\text{target model time}}$

2. E[output tokens] = $\dfrac{1 - \alpha^{k+1}}{1 - \alpha}$

3. round cost = $k * \beta + 1$

4. speedup ratio = $\dfrac{1 - \alpha^{k+1}}{(1 - \alpha) * (k * \beta + 1)}$

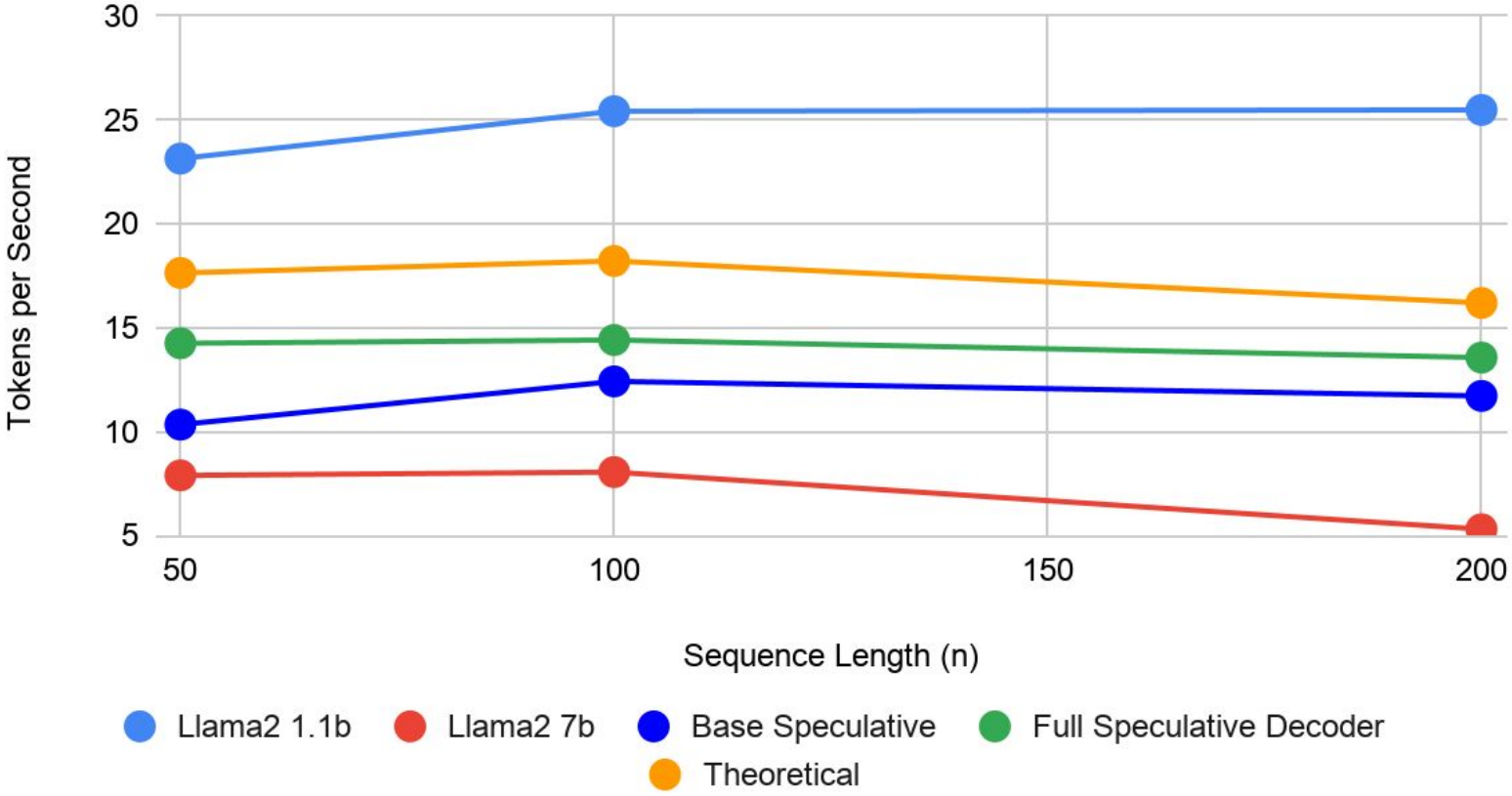
- Using baseline encoders, we find **β** across different sequence lengths **n**
- We generate a table of speedup ratios at different combinations of **α** and **k** for each **n**

Speedup vs Baseline

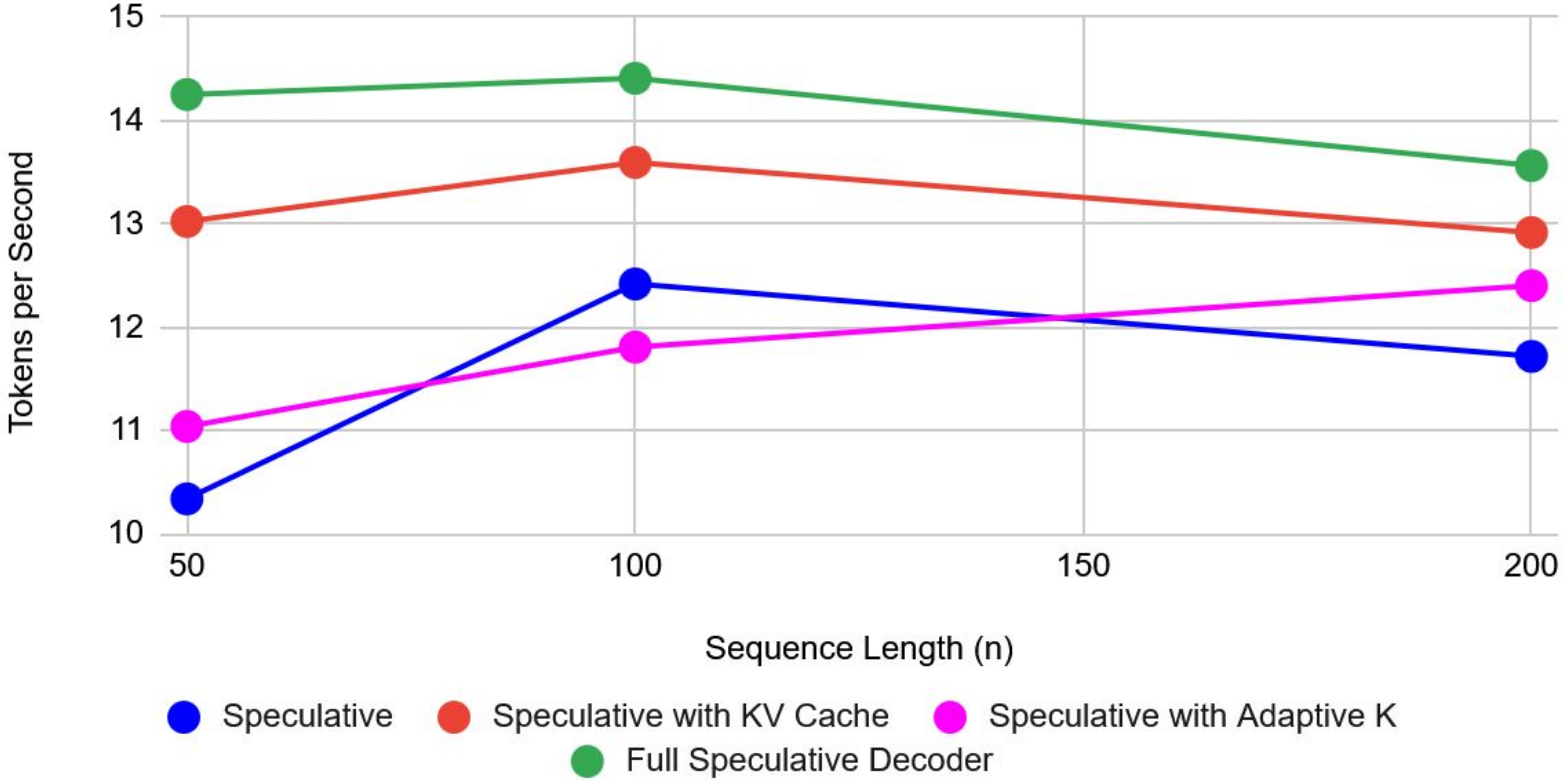
n	β	Theoretical Performance Model Speedup	Actual Speculative Decoder Speed Up
50	0.33	2.2x	1.8x
100	0.25	2.6x	1.8x
200	0.19	3.1x	2.6x

- Performance model was based on Baseline model data
  - Acceptance Rate = 0.6
  - k = 5
- Speculative Decoder implementation underperformed, even with the advantage of Adaptive Speculation Depth

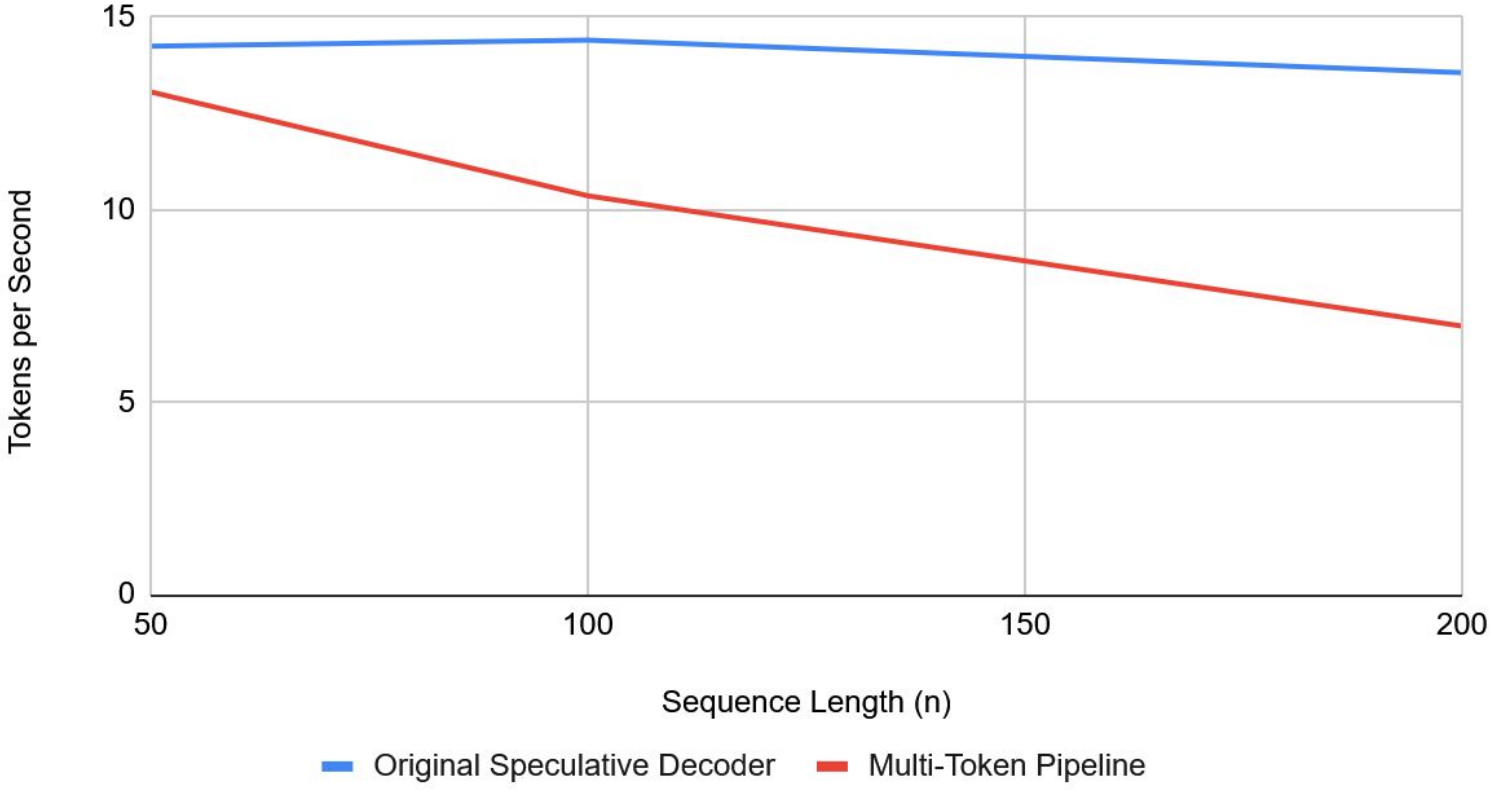
Sequence Length vs Token Throughput



Sequence Length vs Token Throughput with Different Techniques

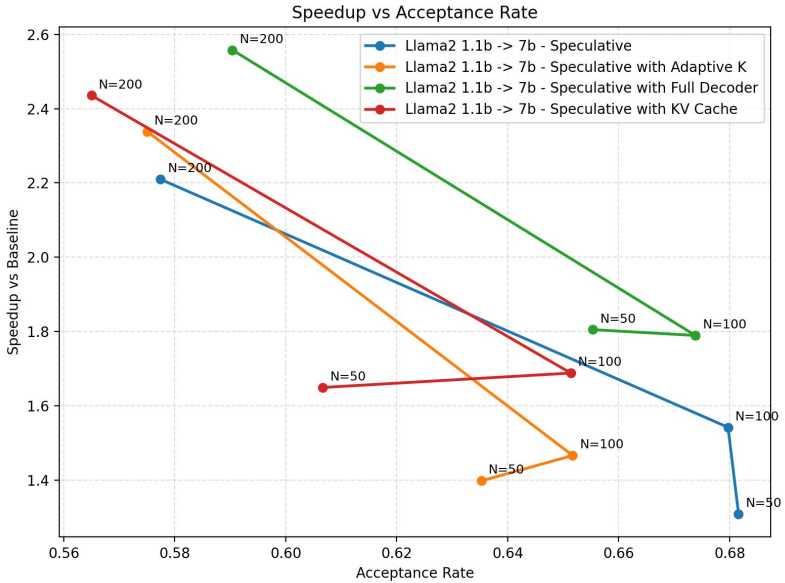


Original Speculative Decoder vs Multi-Token Pipeline



Speedup vs N

- Speed increases with N
- Full Decoder best (~2.5x)
- Shows scaling advantage



Data Analysis

Model	Output (prompt = “The first digits of pi are”)
Llama2 1.1b	The first digits of pi are 3 and 1. The second digit of pi is 1. The third digit of pi is 1. The fourth digit of pi is 1. The fifth digit of pi is 1. The sixth digit of pi is 1.
Llama2 7b	The first digits of pi are 3.14159265358979323846264338327950288419716939937 5
Speculative Decoder	The first digits of pi are 3.14159265358979323846264338327950288419716939937 5

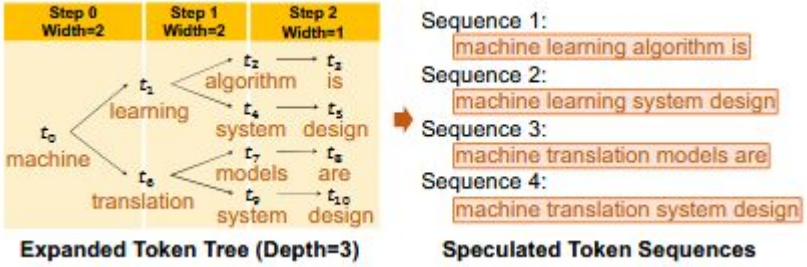
- Draft model, TinyLlama, is fastest as the lightest model
- Full Speculative Decoder with Adaptive K and KV-Caching has best speed with accuracy of the target model
- KV-Caching improves token throughput more than Adaptive K
- Our implementation is underperforming from the theoretical line

Limitations and Failure Modes

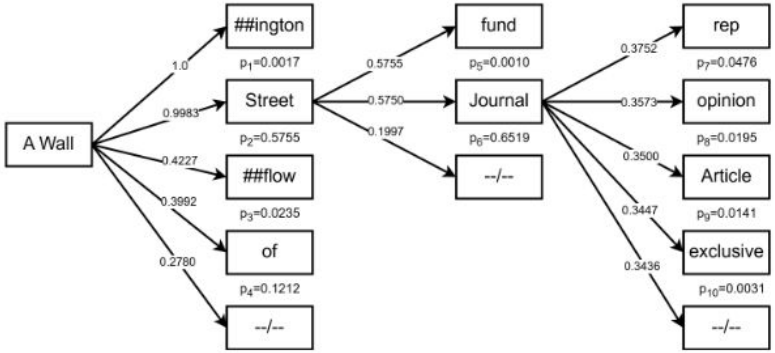
- Our experiment was done on a 1:7 parameter ratio between draft and target, more commonly used is atleast a 1:10 parameter ratio
- More memory storage overhead as device loads two models
  - This is a resource limitation where more VRAM is needed for larger models
- Less factual prompts can lead to low enough acceptance rate where Speculative Decoding is slower than the target model
- The results of the Multi-Token pipeline were excluded due to anomalously low results (~2 t/s), further pursuit in our final paper

Related Research (1)

- SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification (2023)
  - Instead of single sequential chain of draft tokens, build a draft token tree with fixed structure
  - 2.5x to 3.5x speedup

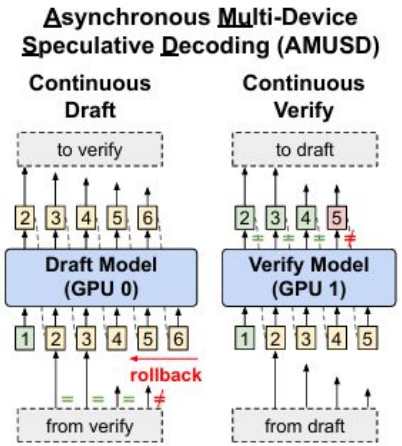


- DySpec: Faster speculative decoding with dynamic token tree structure (2025)
  - Build a dynamically allocated draft token tree based on speculation budget
  - If high confidence, go deeper, if low, then go wider
  - Up to 6.21x speedup

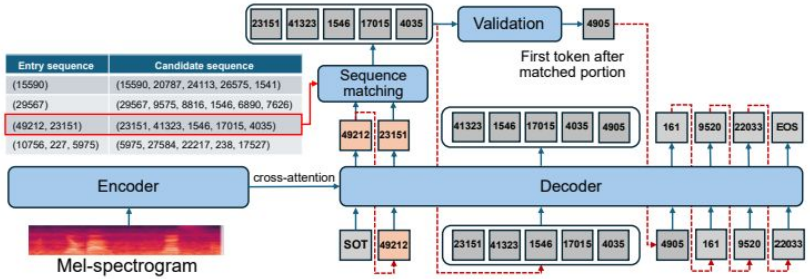


Related Research (2)

- AMUSD: Asynchronous Multi-Device Speculative Decoding for LLM Acceleration (2025)
  - Both draft and target model perform predictions independently across multiple GPUs
  - 29% improvement over speculative decoding with a 1.96x speedup over traditional autoregressive decoding



- Model-free Speculative Decoding for Transformer-based ASR with Token Map Drafting
  - Replaces a draft model with a token map to generate draft tokens
    - Analyzes domain-specific texts to achieve this
  - 1.27x speedup on CI-AVSR dataset and 1.37x on internal dataset



Related Research (3)

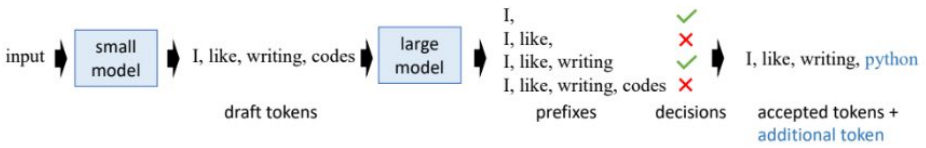


Figure 2: An example of MJSD's verification process. MJSD accepts the *longest* draft sub-sequence that passes verification based on joint likelihood.

- **Decoding Speculative Decoding (Liu et al., 2025)**
- Analyzes speculative decoding performance and bottlenecks
- Shows draft–target mismatch causes frequent rollbacks
- Performance depends on acceptance rate and model alignment
- **Multi-Token Joint Speculative Decoding (Qin et al., 2024)**
- Generates and verifies multiple tokens jointly
- Accepts the longest valid token subsequence
- Achieves higher speedup and lower perplexity than standard speculative decoding

Future Work

- Use models that have greater benefit from speculative decoding
  - Llama2-7B and Llama2-70B (1:10 ratio)
- Explore tree-based drafting and verification
- Parallelize Speculative Decoding process
  - Currently, verification must happen before next draft begins
  - Is there a way to draft before final verification?

Conclusion

- Speculative Decoding can speedup token throughput without cost to accuracy, though constrained
  - Higher memory storage overhead from 2 models loaded, but less memory bandwidth usage bottlenecks as we rely more on computations
  - Models must be agreeable to each other, else low acceptance ratio
  - Choosing a proper k improves speed, and Adaptive Speculation Depth can automate this
- Utilizing either a greedy or random sampling acceptance logic depends on the use case of the prompts
  - High acceptance (math, factual) = Top-n greedy
  - Variable acceptance (opinion) = Random sampling
- KV-Cache improves speed by allowing us to do less computations with little increase in memory bandwidth usage

References

[1] Z. Zhao, Z. Lin, Y. Zhao, Q. Li, G. Huang, and J. Yan, "DySpec: Faster speculative decoding with dynamic token tree structure," arXiv preprint arXiv:2402.05370, 2024.

[2] X. Miao, I. Z. Ganev, J. Wang, C. Cheng, Z. Lin, X. Jia, H. Guo, B. Chen, Z. Wang, Y. Zhu, X. Liu, X. Hu, and C. Wang, "SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification," in Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '24), 2024, pp. 314–329.

[3] T. V. Ho, H. Kokubo, M. Yamamoto and Y. Kawaguchi, "Model-Free Speculative Decoding for Transformer-Based ASR with Token Map Drafting," 2025 33rd European Signal Processing Conference (EUSIPCO), Palermo, Italy, 2025, pp. 361-365, doi: 10.23919/EUSIPCO63237.2025.11226729.

[4] B. McDanel, "AMUSD: Asynchronous Multi-Device Speculative Decoding for LLM Acceleration," 2025 IEEE International Symposium on Circuits and Systems (ISCAS), London, United Kingdom, 2025, pp. 1-5, doi: 10.1109/ISCAS56072.2025.11043575.

[5] Z. Qin et al., “Multi-Token Joint Speculative Decoding for Accelerating Large Language Model Inference,” arXiv preprint arXiv:2407.09722, 2024.

[6] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding Speculative Decoding," in Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2025, pp. 6460–6473.

Questions?